

RETO TÉCNICO

QA AUTOMATION LEAD

ACME Corp

Estrategia integral de calidad: Envío de Mensajes y Envío de Archivos Adjuntos

Autor: Elvis Mieles Mieles

Fecha: 11 de September de 2026

Tabla de Contenido

Actualice la tabla de contenido (clic derecho > Actualizar campo) al abrir el documento.

1. Resumen Ejecutivo

ACME Corp está evolucionando su plataforma de mensajería hacia un volumen masivo de uso (miles de usuarios, millones de mensajes diarios en un horizonte de 6 meses). Esta propuesta de estrategia de QA se enfoca en dos funcionalidades de mayor riesgo y valor demostrativo: envío de mensajes (núcleo del producto, con estados y tiempo real) y envío de archivos adjuntos (complejidad de seguridad y procesamiento de datos no estructurados).

La estrategia aplica Risk-Based Testing para priorizar casos críticos (duplicidad, pérdida silenciosa de mensajes, seguridad en uploads, control de acceso) como P0, ejecutándose de forma automatizada desde el Pull Request hasta el smoke post-deploy. Se adopta una pirámide de automatización (Unit → API → Integration → UI mínimo → E2E crítico) para mantener una suite rápida, estable y mantenible, evitando la sobre-automatización en áreas de bajo retorno (usabilidad, exploratorio, pentesting creativo).

El pipeline de CI/CD se organiza en cinco niveles (PR, merge, QA/Staging, pre-producción, post-deployment), balanceando velocidad de feedback con profundidad de cobertura — un factor crítico dado el ritmo de sprints de una semana. Shift Left se incorpora desde el refinamiento, con un checklist de QA que evalúa testabilidad, riesgos y requisitos no funcionales antes de codificar.

Ante el crecimiento proyectado, se propone una incorporación progresiva de pruebas de performance, concurrencia, resiliencia y observabilidad durante los próximos 6 meses, con SLO/SLI a definir junto al negocio. Se establecen Quality Gates por etapa y métricas orientadas a decisiones (defect leakage, MTTD/MTTR, flaky rate), no solo conteo de ejecuciones.

Todos los supuestos técnicos (arquitectura, tecnología de tiempo real, límites de archivo) están explícitamente marcados como pendientes de validación con Arquitectura/Product Owner, evitando presentarlos como requisitos confirmados de ACME.

2. Selección de Funcionalidades

De las cinco funcionalidades propuestas por ACME Corp (envío de mensajes, mensajes editados, mensajes eliminados, reacciones a mensajes, envío de archivos adjuntos), se seleccionaron dos que en conjunto permiten demostrar una estrategia de QA integral, cubriendo ejes de riesgo distintos y complementarios.

Funcionalidad	Riesgo	Complejidad	Automatización	Impacto negocio	Valor QA Lead
Envío de mensajes	Crítico	Alto	Alto	Crítico	Alto
Envío de archivos adjuntos	Alto	Crítico	Medio	Alto	Alto
Mensajes editados	Medio	Medio	Alto	Medio	Medio
Reacciones a mensajes	Bajo	Bajo	Alto	Bajo	Bajo
Mensajes eliminados	Medio	Medio	Medio	Medio	Medio

Funcionalidades seleccionadas: Envío de Mensajes y Envío de Archivos Adjuntos.

Envío de mensajes representa el flujo núcleo del producto, con estados (enviado, entregado, fallido, procesando), fuerte dependencia de tiempo real, concurrencia y escalabilidad — ideal para demostrar State Transition Testing y estrategias de performance a gran escala. Envío de archivos adjuntos aporta un eje de riesgo distinto: seguridad (validación de tipos, path traversal), procesamiento de datos no estructurados y dependencias de almacenamiento externo. Juntas cubren Functional, Integration, Performance, Security, Resilience y State Transition Testing sin solaparse, demostrando un criterio de selección basado en cobertura de riesgo diversificada.

3. Funcionalidad 1 — Envío de Mensajes

3.1 Análisis

Requisito: ACME Corp está implementando/modificando la funcionalidad de envío de mensajes.

Supuestos y gaps identificados (no confirmados por ACME):

- Supuesto: existe un backend con API (REST y/o WebSocket) que recibe el mensaje, lo persiste y lo distribuye a los destinatarios.
- Supuesto: existen estados de mensaje: enviado, procesando, entregado, fallido.
- Supuesto técnico / punto a validar: uso de un canal en tiempo real (WebSocket u otro) para notificar entrega.
- Supuesto: los mensajes pueden enviarse en chats 1:1 y/o grupales.
- Gap: no se especifica límite de longitud de mensaje, rate limiting, ni SLA de entrega.
- Pregunta abierta: ¿existe garantía de orden de mensajes por conversación? ¿se permite edición/reenvío automático ante fallos?

3.2 Escenarios de Prueba

Happy Path

#	Escenario	Capa
HP1	Usuario autenticado envía mensaje de texto válido a un contacto activo y lo recibe correctamente	E2E/API
HP2	Usuario envía mensaje en un chat grupal y todos los miembros lo reciben	Integration
HP3	Mensaje pasa correctamente por los estados: procesando → enviado → entregado	State Transition

Negative Testing

- N1 — Envío de mensaje vacío o solo espacios en blanco.
- N2 — Envío a un destinatario inexistente o eliminado.
- N3 — Envío sin token de autenticación válido / sesión expirada.
- N4 — Envío con payload malformado (JSON inválido).
- N5 — Servidor responde 5xx durante el envío.
- N6 — Usuario bloqueado intenta enviar mensaje a quien lo bloqueó.

Boundary Value Analysis (supuesto: existe un límite de caracteres por mensaje)

- Mensaje con longitud igual al límite máximo permitido.
- Mensaje con longitud = límite + 1.
- Mensaje con 1 solo carácter.
- Mensaje con emojis/Unicode en el límite.

Equivalence Partitioning

- Clase válida: texto plano estándar.
- Clase válida: texto con emojis/Unicode.
- Clase inválida: texto con scripts/HTML (posible XSS).
- Clase inválida: caracteres nulos o binarios.

State Transition Testing

Estados: procesando → enviado → entregado; procesando → fallido; fallido → reintento → enviado. Se valida que no existan transiciones inválidas (p. ej., entregado → procesando) y el comportamiento ante timeout de confirmación (Gap: SLA no definido).

Integration, Compatibility, Concurrency y Recovery

- Integration: Frontend↔API, API↔Base de datos, API↔notificaciones push, API↔canal en tiempo real (supuesto técnico), Backend↔servicio de presencia.
- Compatibility: navegadores principales, apps móviles (si aplica — gap), resoluciones y orientaciones, condiciones de red variables.
- Concurrency: envíos simultáneos en el mismo chat, miles de usuarios enviando a la vez, múltiples dispositivos de una misma cuenta.
- Recovery: pérdida de conexión durante el envío, timeout sin duplicados, reconexión de WebSocket con sincronización de pendientes, caída temporal del servicio de mensajería.

3.3 Casos Críticos

#	Caso	Impacto usuario/negocio	Capa	¿Automatizar?	Momento pipeline	Prioridad
C1	Duplicidad de mensajes bajo pérdida de conexión/retry	Confusión; percepción de bug grave; soporte saturado	API/Integration	Sí	Integración + regresión	P0
C2	Pérdida silenciosa de mensajes (queda en 'procesando')	Comunicación perdida sin aviso; riesgo crítico de negocio	Integration/E2E	Sí	Regresión + smoke post-deploy	P0
C3	Orden incorrecto de mensajes en concurrencia alta	Confusión, malentendidos; impacto en confianza	Integration/Performance	Sí (parcial)	Regresión/carga	P1
C4	Envío exitoso pese a autenticación inválida/expirada	Riesgo de suplantación; incumplimiento de seguridad	API/Security	Sí	PR checks + seguridad	P0
C5	Degradación de latencia con miles de usuarios simultáneos	Percepción de app lenta/rota; churn	Performance	Sí	Pipeline de performance	P1
C6	Mensaje con contenido malicioso (XSS) se renderiza sin sanitizar	Vulnerabilidad explotable; riesgo legal	Security	Sí	Seguridad + PR checks	P0
C7	Falla al reconectar WebSocket no sincroniza mensajes pendientes	Usuario pierde contexto de conversación	E2E/Recovery	Parcial	Regresión	P1

Fórmula de priorización: Impacto × Probabilidad × Detectabilidad. Los casos con impacto crítico y baja detectabilidad manual (duplicidad, pérdida silenciosa, seguridad) se clasifican P0 y se ejecutan automatizados en etapas tempranas, además de siempre en regresión/smoke post-deploy.

3.4 Matriz de Riesgos

Riesgo	Probabilidad	Impacto	Prioridad
Duplicidad de mensajes	Media	Crítico	P0
Pérdida silenciosa de mensajes	Media	Crítico	P0
XSS en contenido de mensaje	Media	Crítico	P0
Autenticación inválida no bloqueada	Baja	Crítico	P0
Degradación de latencia en concurrencia	Alta	Alto	P1
Orden incorrecto en concurrencia	Media	Alto	P1
Fallo de sincronización tras reconexión	Media	Medio	P1
Incompatibilidad en navegadores/dispositivos	Baja	Medio	P2

3.5 Automatización vs Manual

Tipo de prueba	Automatizar	Manual	Justificación
Unit tests	Sí	-	Lógica de estados y validaciones de payload; rápido y determinístico.
API tests	Sí	-	Envío/recepción determinístico y repetitivo; ideal para CI.
Integration tests	Sí	-	API↔DB↔notificaciones; crítico y estable una vez definido el contrato.
Contract tests	Sí	-	Evita romper integraciones si frontend/backend evolucionan por separado.
UI tests	Parcial	Parcial	Solo 2-3 flujos críticos automatizados; resto exploratorio.
E2E	Parcial	-	Solo flujo crítico completo; mantener suite mínima.
Regression	Sí	-	Repetitivo y crítico en cada release.
Smoke	Sí	-	Rápido y determinístico; siempre post-deploy.
Sanity	Sí	-	Subconjunto rápido tras hotfixes.
Exploratory	-	Sí	UX, edge cases no anticipados, red inestable.
Accessibility	Parcial	Sí	Checks básicos automatizables; experiencia real manual.
Performance	Sí	-	Determinístico; pipeline dedicado, no en cada PR.

Security	Parcial	Sí	Escaneo estático automatizado; pentesting avanzado manual.
Compatibility	Parcial	Sí	Smoke en navegadores principales; resto manual.
Chaos/resilience	Parcial	Sí	Básico automatizado; pruebas profundas controladas y manuales.

No se automatizarían: usabilidad y percepción de 'app lenta' (juicio subjetivo), exploratory testing de reconexión bajo condiciones reales de red (bajo ROI), pentesting avanzado (requiere creatividad no repetible) y casos raros de compatibilidad con dispositivos legacy (bajo volumen de usuarios afectados).

3.6 Estrategia de Automatización

- Nivel Unitario (Development): reglas de negocio, longitud de mensaje, sanitización de input, transición de estados.
- Nivel API (QA Automation): envío válido/inválido, autenticación, autorización, idempotencia en reintentos, estados, rate limiting.
- Nivel Integration: API↔DB, API↔notificaciones, API↔canal en tiempo real, comportamiento ante caída de un servicio dependiente.
- Nivel UI: 2-3 flujos (envío exitoso, error visible, estado 'entregado' reflejado).
- Nivel E2E: un único flujo crítico de negocio (A envía, B recibe en tiempo real, estado se actualiza).

Para evitar una suite lenta, frágil, duplicada y costosa: pirámide de pruebas (mayoría en Unit/API), datos de prueba aislados y reseteables, ejecución en paralelo, y revisión periódica de tests flaky/obsoletos cada sprint.

3.7 Priorización

- P0 (duplicidad, pérdida silenciosa, XSS, auth bypass): PR/merge cuando sea posible, y siempre en smoke post-deploy.
- P1 (conurrencia, orden, reconexión): pipeline de integración/regresión.
- P2 (compatibilidad): pipeline de regresión completa, menor frecuencia.

3.8 CI/CD

Etapas	Pruebas incluidas
PR check	Unit tests, linters, validaciones de sanitización básica
Merge a main	API tests (happy path + negativos críticos), contract tests
Deploy a QA/Staging	Integration tests, smoke, regresión seleccionada (P0/P1)
Pre-producción	Regresión completa, performance básico, security scan
Post-deployment	Smoke E2E crítico, monitoreo de error rate en tiempo real

4. Funcionalidad 2 — Envío de Archivos Adjuntos

4.1 Análisis

Requisito: ACME Corp está implementando/modificando la funcionalidad de envío de archivos adjuntos.

- Supuesto: existe un servicio de almacenamiento (local o cloud) donde se guardan los archivos.
- Supuesto: existen validaciones de tipo y tamaño de archivo (no especificadas por ACME).
- Supuesto técnico / punto a validar: el upload puede ser síncrono o asíncrono con progreso.
- Supuesto: existe algún mecanismo de escaneo de seguridad (antivirus/malware) antes de distribuir el archivo.
- Gap: no se especifica límite máximo de tamaño de archivo, tipos permitidos, ni política de retención.
- Gap: no se especifica si existe compresión/generación de miniaturas.
- Pregunta abierta: ¿los archivos se cifran en tránsito y en reposo? ¿existe control de acceso granular?

4.2 Escenarios de Prueba

Happy Path

#	Escenario	Capa
HP1	Usuario autorizado adjunta un archivo válido y se envía correctamente	E2E/API
HP2	Archivo se sube, se almacena y el destinatario puede descargarlo/visualizarlo íntegro	Integration
HP3	Progreso de subida se refleja correctamente (0% → 100% → completado)	UI

Negative Testing

- N1 — Subida de archivo con extensión no permitida (.exe, .bat, etc.).
- N2 — Subida de archivo que excede el tamaño máximo permitido.
- N3 — Subida de archivo vacío (0 bytes) o corrupto.
- N4 — Cancelación de subida a mitad de proceso.
- N5 — Fallo del servidor de almacenamiento durante el upload (5xx).
- N6 — Subida sin permisos/autenticación válida.
- N7 — Nombre de archivo con caracteres especiales/path traversal.

Boundary Value Analysis / Equivalence Partitioning

- Archivo en el límite máximo permitido, límite+1 byte, y tamaño mínimo (1 byte).
- Clases válidas: imágenes (jpg, png), documentos (pdf, docx).
- Clases inválidas: ejecutables/scripts, archivos con doble extensión.

State Transition, Integration, Compatibility, Concurrency y Recovery

- Estados: subiendo → procesando → disponible; subiendo → fallido → reintento. Validar que archivos fallidos/cancelados no queden referenciados ni dejen residuos (orphan files).
- Integration: API↔almacenamiento, API↔DB (metadata), API↔escaneo de seguridad, API↔notificaciones, generación de thumbnails.
- Compatibility: drag&drop en navegadores, subida desde cámara/galería móvil (gap), formatos entre sistemas operativos.

- **Concurrency:** múltiples usuarios subiendo simultáneamente, un usuario con múltiples uploads en paralelo, miles de uploads a nivel de sistema.
- **Recovery:** pérdida de conexión en archivos grandes sin duplicar/corromper, timeout con mensaje claro, recuperación de storage sin pérdida de archivos confirmados.

4.3 Casos Críticos

#	Caso	Impacto usuario/negocio	Capa	¿Automatizar?	Momento pipeline	Prioridad
C1	Subida de archivo malicioso sin bloqueo	Vector de ataque directo; riesgo legal y reputacional severo	Security/API	Sí	PR + seguridad	P0
C2	Path traversal en nombre de archivo	Acceso/sobrescritura de archivos del sistema	Security/API	Sí	PR + seguridad	P0
C3	Archivo corrupto marcado como disponible	Descarga inutilizable; pérdida de confianza	Integration	Sí	Regresión + smoke	P0
C4	Archivos huérfanos tras cancelación/fallo	Costos crecientes de almacenamiento a escala	Integration	Sí	Regresión nocturna	P1
C5	Timeout indefinido en archivos grandes bloquea la UI	Frustración, abandono	E2E/Performance	Sí (parcial)	Pipeline performance	P1
C6	Degradación de storage/ancho de banda con uploads masivos	Subidas lentas/fallidas en hora pico; churn	Performance	Sí	Pipeline performance	P1
C7	Falta de control de acceso permite descarga no autorizada	Exposición de datos privados; riesgo legal	Security/API	Sí	PR + seguridad	P0

4.4 Matriz de Riesgos

Riesgo	Probabilidad	Impacto	Prioridad
Subida de archivo malicioso	Media	Crítico	P0
Path traversal / nombre inseguro	Baja	Crítico	P0
Acceso no autorizado a archivos	Media	Crítico	P0
Archivo corrupto marcado como disponible	Media	Alto	P0
Archivos huérfanos (costo de almacenamiento)	Alta	Medio	P1
Timeout/bloqueo de UI en archivos grandes	Media	Alto	P1

Degradación bajo carga masiva de uploads	Alta	Alto	P1
Incompatibilidad de formatos entre dispositivos	Baja	Medio	P2

4.5 Automatización vs Manual

Tipo de prueba	Automatizar	Manual	Justificación
Unit tests	Sí	-	Reglas de tipo/tamaño/nombre de archivo.
API tests	Sí	-	Upload/download determinístico; permisos y metadata.
Integration tests	Sí	-	API↔Storage↔DB↔escaneo de seguridad; crítico y repetible.
Contract tests	Sí	-	Protege contra cambios de contrato si storage es externo.
UI tests	Parcial	Parcial	Drag&drop y progreso automatizados; resto exploratorio.
E2E	Parcial	-	Solo flujo crítico: adjuntar → subir → recibir → descargar íntegro.
Regression	Sí	-	Repetitivo y crítico en cada release.
Smoke	Sí	-	Disponibilidad del storage tras deploy.
Exploratory	-	Sí	Archivos raros, redes inestables, cámara/galería.
Performance	Sí	-	Uploads grandes/concurrentes medibles y repetibles.
Security	Parcial	Sí	Validación de tipo/path automatizada; pentesting manual.
Compatibility	Parcial	Sí	Smoke en formatos principales; resto manual.
Chaos/resilience	Parcial	Sí	Corte de conexión simulado automatizado; caída real de storage manual/periódica.

No se automatizarían: experiencia de subida desde cámara/galería móvil (alta dependencia de hardware real), percepción de 'app lenta' durante upload (subjetivo), pentesting avanzado de bypass de validación (creatividad adversarial) y casos raros de compatibilidad de formatos legacy (bajo ROI).

4.6 Estrategia de Automatización

- Nivel Unitario: reglas de tipo, extensión, tamaño máximo, sanitización de nombre.
- Nivel API: upload válido/inválido, permisos de descarga, metadata persistida, archivo corrupto/cancelado.
- Nivel Integration: API↔Storage real, API↔escaneo de seguridad, API↔notificaciones, limpieza de huérfanos.
- Nivel UI: adjuntar archivo válido y ver progreso/confirmación; error claro ante archivo inválido.
- Nivel E2E: adjuntar → subir → recibir → descargar íntegro (checksum).

Para evitar una suite lenta y costosa: archivos de prueba pequeños y controlados, mock del storage en pruebas rápidas de integración (storage real solo en staging), y no repetir validaciones de tipo/tamaño en UI ya cubiertas en Unit/API.

4.7 Priorización

- P0 (malware, path traversal, acceso no autorizado, corrupción): PR/merge y siempre en smoke/regresión post-deploy.
- P1 (huérfanos, timeout, degradación bajo carga): pipeline de integración/performance.
- P2 (compatibilidad de formatos/dispositivos): regresión completa periódica.

4.8 CI/CD

Etapas	Pruebas incluidas
PR check	Unit tests de validación de tipo/tamaño/nombre, escaneo estático de seguridad
Merge a main	API tests (upload/download, permisos, path traversal), contract tests
Deploy a QA/Staging	Integration con storage real, smoke, regresión seleccionada (P0/P1)
Pre-producción	Regresión completa, performance de uploads concurrentes, security scan
Post-deployment	Smoke E2E crítico (upload→disponible→descarga íntegra), monitoreo de fallos

5. Estrategia CI/CD

Principio guía: no todo se ejecuta en cada pipeline; el tiempo de feedback debe ser proporcional al riesgo y al costo de ejecución.

Nivel de Pipeline	Objetivo	Pruebas incluidas	Duración objetivo
Pipeline rápido (PR)	Feedback en minutos	Unit tests, linters/static analysis, sanitización (XSS, path traversal), API tests PO	< 5-8 min
Pipeline de integración (merge)	Validar integración real	API tests completos, integration tests, contract tests	< 15-20 min
Pipeline de regresión (QA/Staging)	Mayor cobertura funcional	Regresión P0/P1 completa, smoke, integration extendida, security scan	20-40 min
Pipeline pre-producción	Garantizar readiness	Regresión completa, performance/carga, security scan profundo, compatibility smoke	Ventana nocturna
Post-deployment	Verificar salud en vivo	Smoke E2E crítico, sanity, monitoreo de error rate/latencia	Minutos + alertas

Ejecutar performance/regresión completa en cada PR generaría una suite insostenible en sprints de 1 semana; por eso se reservan a etapas de mayor consolidación.

6. Gestión y Mantenimiento de Casos de Prueba

Proceso propuesto: Idea → Refinamiento → Riesgos → Criterios de aceptación → Casos → Automatización → CI/CD → Ejecución → Evidencia → Métricas → Mantenimiento.

- Refinement: QA participa desde el inicio para asegurar testabilidad antes de escribir casos.
- Definition of Ready/Done: criterios claros evitan iniciar desarrollo sin casos de aceptación testeables.
- Traceability: cada caso se enlaza a un requisito y a un riesgo.
- Test Case Management: repositorio único (herramienta a definir con el equipo — Gap), versionado junto al código.
- Review: casos nuevos/modificados se revisan en PR como cualquier artefacto de código.
- Ownership: QA Automation dueño de la suite automatizada; Developers de unit tests; QA manual dueño de exploratorio.
- Deprecación: revisión periódica de tests flaky o redundantes cada sprint.
- Métricas: usadas para decidir qué mantener, refactorizar o eliminar.

7. QA desde Refinamiento (Shift Left)

Checklist de QA para Refinement:

- Reglas de negocio claramente definidas y sin ambigüedad.
- Criterios de aceptación medibles y testeables.
- Casos límite (edge cases) identificados junto al equipo.
- Casos negativos considerados (no solo el happy path).
- Dependencias externas identificadas.
- Riesgos de negocio y técnicos discutidos abiertamente.

- Testabilidad validada (¿se puede automatizar? ¿hay entornos disponibles?).
- Observabilidad considerada (logs/métricas necesarias para validar en producción).
- Datos de prueba requeridos identificados con anticipación.
- Factibilidad de automatización evaluada antes de comprometer el sprint.
- Requisitos no funcionales (performance, seguridad, accesibilidad) discutidos, no asumidos.
- Compatibilidad (browser/mobile/OS) considerada si aplica.

Definition of Ready for Testing (Propuesta)

Nota: propuesta a acordar con el equipo, no un requisito confirmado de ACME.

- Historia desarrollada y desplegada en ambiente de prueba.
- Criterios de aceptación definidos y aprobados.
- Ambiente estable disponible.
- Datos de prueba preparados y reseteables.
- API documentada (contratos claros de entrada/salida).
- Feature flag configurado, si aplica.
- Logs/trazas disponibles para depuración.
- Dependencias externas disponibles o mockeadas.
- Unit tests ejecutados y en verde.
- Evidencia técnica (build, changelog) disponible para QA.

8. Escalabilidad — Millones de Mensajes (próximos 6 meses)

La estrategia de QA debe evolucionar de 'validar funcionalidad' a 'validar comportamiento bajo escala', incorporando pruebas de forma progresiva:

Mes	Enfoque incremental
1-2	Load testing básico (cientos → miles de usuarios simulados), definición de SLO/SLI iniciales
3-4	Stress testing (punto de quiebre), endurance testing (24-72h sostenidas)
5	Spike testing (picos súbitos de tráfico), chaos/resilience testing controlado
6	Capacity testing (límites de escalado horizontal), simulacros de recuperación ante desastres

- Performance: load, stress, spike, endurance y capacity testing sobre los flujos críticos priorizados.
- Concurrencia: miles de usuarios enviando mensajes/archivos simultáneamente; validar orden, idempotencia y ausencia de duplicados a escala.
- Backend: comportamiento de APIs, base de datos, colas y cachés bajo saturación (supuesto técnico: microservicios/colas/event-driven no confirmados; validar con Arquitectura).
- Mensajería en tiempo real: si se confirma WebSocket/eventos, incorporar pruebas de reconexión masiva y entrega garantizada bajo carga.
- Observabilidad: logs estructurados, métricas, trazas distribuidas y alertas proactivas.
- Datos: retención, consistencia y ausencia de duplicidad/corrupción a escala.
- Resiliencia: timeout, retry con backoff, failover, degradación controlada y recuperación automática sin pérdida de datos.

SLO/SLA/SLI propuestos (a validar con negocio): latencia de entrega (p95/p99), error rate máximo aceptable, throughput sostenido, availability (%), message delivery rate.

9. Quality Gates

Propuesta a acordar con el equipo; los valores numéricos no están confirmados por ACME.

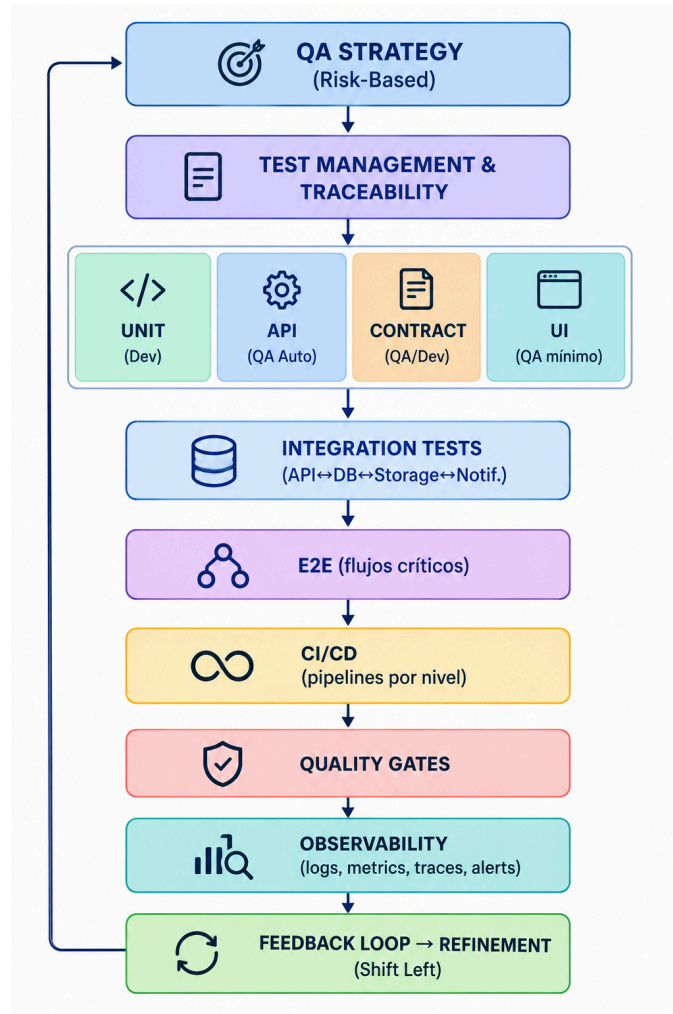
Etapas	Criterio propuesto
Pull Request	0 fallos en unit tests; linter sin errores; escaneo estático sin vulnerabilidades críticas
CI (merge)	0 fallos en API/integration tests P0; contract tests en verde
QA Environment	Smoke 100% exitoso; 0 defectos P0 abiertos
Staging	Regresión crítica (P0/P1) 100% exitosa; performance dentro del SLO propuesto; security scan sin críticos
Production	0 defectos P0 abiertos; smoke post-deploy 100% exitoso; monitoreo sin alertas críticas iniciales

10. Métricas de QA

Métrica	Uso para toma de decisiones
Automation coverage	Identificar áreas sub-automatizadas para priorizar inversión
Critical path coverage	Asegurar que los flujos P0 estén siempre cubiertos
Pass/Failure rate	Detectar tendencias de inestabilidad en el pipeline
Defect leakage / escape rate	Medir efectividad real de QA antes de producción

Defect density	Identificar módulos de mayor riesgo
MTTD / MTTR	Evaluar velocidad de respuesta ante incidentes en producción
Flaky test rate	Priorizar estabilización de la suite
Pipeline/regression duration	Detectar cuándo optimizar/paralelizar
Requirements/risk coverage	Confirmar que cada riesgo tiene un caso asociado
Production incidents	Retroalimentar la estrategia de testing

11. Arquitectura de Automatización



Se agrega el feedback loop hacia refinement, cerrando el ciclo entre lo detectado en producción/observabilidad y la mejora continua de historias futuras — reforzando que QA es transversal, no lineal.

12. Matriz de Trazabilidad

Funcionalidad 1 — Envío de Mensajes

Requirement	Risk	Scenario	Test Case	Automation	CI/CD	Priority
Envío de mensaje válido	Pérdida silenciosa	HP1	Enviar y validar entrega	Sí (API/E2E)	Merge + Smoke	P0
Reintento ante fallo de red	Duplicidad	Recovery	Reintento sin duplicar	Sí (API)	Integración	P0
Sanitización de contenido	XSS	Equivalence Part.	Envío de script embebido	Sí (Security/API)	PR check	P0
Validación de sesión	Auth bypass	N3	Envío con token inválido	Sí (API)	PR check	P0
Envío concurrente	Orden incorrecto	Concurrency	Envíos simultáneos	Sí (Integration)	Regresión	P1
Reconexión en tiempo real	Sincronización fallida	Recovery	Reconexión tras pérdida WS	Parcial	Regresión	P1

Funcionalidad 2 — Envío de Archivos Adjuntos

Requirement	Risk	Scenario	Test Case	Automation	CI/CD	Priority
Validación de tipo de archivo	Archivo malicioso	N1	Subida de .exe rechazada	Sí (Unit/API)	PR check	P0
Sanitización de nombre de archivo	Path traversal	N7	Nombre con '../' rechazado	Sí (Security/API)	PR check	P0
Control de acceso a descarga	Acceso no autorizado	Integration	Descarga sin permisos bloqueada	Sí (API)	PR check	P0
Integridad de archivo	Corrupción	Recovery	Checksum post-upload	Sí (Integration)	Integración	P0
Limpieza de archivos huérfanos	Costo de almacenamiento	State Transition	Cancelación sin residuos	Sí (Integration)	Regresión nocturna	P1
Uploads concurrentes masivos	Degradación de storage	Concurrency	Miles de uploads simultáneos	Sí (Performance)	Pipeline performance	P1

13. Riesgos y Gaps Identificados

ID	Tipo	Descripción	Impacto	Probabilidad	Mitigación	Responsable
R1	Assumption	Arquitectura de tiempo real (WebSocket/eventos) no confirmada	Alto	Media	Validar con Arquitectura antes de automatizar integración	Arquitectura/QA Lead
R2	Gap	SLA/SLO numéricos no definidos (latencia, delivery rate)	Alto	Alta	Acordar valores en próximo refinamiento con negocio	Product Owner/QA Lead
R3	Gap	Herramienta de Test Case Management no seleccionada	Medio	Alta	Seleccionar herramienta versionable junto al código	QA Lead

R4	Assumption	Política de retención/cifrado de archivos no confirmada	Alto	Media	Validar con Seguridad/Arquitectura	Seguridad/Arquitectura
R5	Open Question	Existencia de app móvil no confirmada	Medio	Media	Confirmar con Product Owner para ajustar compatibility testing	Product Owner
R6	Risk	Suite E2E frágil si no se gestionan datos de prueba aislados	Medio	Media	Test Data Management dedicado y limpieza automática	QA Automation
R7	Requirement	Envío de mensajes y archivos adjuntos confirmados como alcance del reto	-	-	N/A — información proporcionada por ACME	N/A

14. Conclusión

Esta estrategia demuestra que QA no es una etapa al final del desarrollo, sino una responsabilidad transversal durante todo el ciclo de vida del producto: desde el refinamiento de la historia (Shift Left), pasando por el diseño de una suite de automatización balanceada y sostenible (pirámide de pruebas), hasta la observabilidad en producción que retroalimenta las decisiones de las próximas historias. El enfoque Risk-Based permite priorizar el esfuerzo de calidad donde el impacto de negocio es mayor, mientras que la diferenciación explícita entre requisitos, supuestos, gaps y preguntas abiertas asegura que la propuesta sea honesta, trazable y accionable para ACME Corp, incluso con información incompleta sobre su arquitectura técnica.